Phase 7: Adaptive Behaviour

This document provides the complete implementation for **Phase 7: Adaptive Behaviour** for the **Banking — AI Banking Support & Advisory Agent (Non-Transactional)** project. In earlier phases, the banking AI assistant gained: • LLM reasoning • Retrieval-Augmented Generation (RAG) • Tool usage • Multi-turn memory • Planning and contextual understanding Phase 7 improves the system further by introducing adaptive behaviour. The banking AI assistant can now: • Collect user feedback • Store feedback safely • Modify future responses • Improve personalization • Adapt communication style • Explain behavioural changes

Banking Safety Constraints

- The system must never adapt into unsafe banking behaviour.
- User feedback cannot override compliance rules.
- The agent must never learn transactional actions.
- PII must not be stored in feedback systems.
- Unsafe or abusive feedback must be ignored.
- High-risk banking rules remain fixed and non-adaptive.

Why Adaptive Behaviour is Needed

Earlier versions of the banking AI assistant used static behaviour. This caused several usability problems: • Responses felt repetitive. • The system could not learn preferred communication styles. • The assistant repeated explanations unnecessarily. • User dissatisfaction could not influence future interactions. • Personalized support was limited. Phase 7 introduces controlled behavioural adaptation while preserving banking safety.

1. Adaptive Behaviour Architecture

User Query ↓ Conversation + Memory Context ↓ Feedback Store Lookup ↓ Behaviour Adaptation Layer ↓ LLM Response Generation ↓ Safety Validation ↓ Final Banking Response

2. Collecting User Feedback

The banking assistant now collects explicit user feedback. Examples: • "This answer was helpful." • "Please give shorter responses." • "I prefer simpler explanations." • "Use more formal language."

```
feedback_store = []

def save_feedback(user_id, feedback):

    feedback_store.append({
        "user_id": user_id,
        "feedback": feedback
    })

save_feedback(
    user_id="user_101",
    feedback="Prefer shorter explanations"
)

print(feedback_store)
```

3. Supported Feedback Signals

- Response length preference
- Preferred tone (formal/simple)
- Preferred banking topics
- Response helpfulness ratings
- Repeated dissatisfaction signals
- Escalation preferences

4. Behaviour Adaptation Logic

The adaptation layer modifies future responses using stored feedback. Example adaptations: • Shorter responses for users preferring concise answers • Simpler language for non-technical users • Formal tone for professional customers • More detailed fraud guidance for users frequently asking fraud questions

```
def get_response_style(feedback_list):

    style = {
        "tone": "neutral",
        "length": "medium"
    }

    for item in feedback_list:

        feedback = item["feedback"].lower()

        if "short" in feedback:
            style["length"] = "short"

        if "formal" in feedback:
            style["tone"] = "formal"

    return style
```

5. Dynamic Prompt Adaptation

Stored feedback dynamically changes the system prompt before generating responses.

```
def build_system_prompt(style):

    prompt = '''
    You are a Banking AI Assistant.
    Follow banking compliance rules.
    '''

    if style["length"] == "short":
        prompt += "\nGive concise answers."

    if style["tone"] == "formal":
        prompt += "\nUse professional formal tone."

    return prompt
```

6. Before vs After Behaviour

The following example demonstrates behavioural adaptation. User Feedback: "Please keep responses short and formal."

Scenario	Before Adaptation	After Adaptation
Credit card query	Long conversational explanation	Concise professional response
Fraud reporting	Detailed step-by-step discussion	Short formal escalation guidance
Loan information	Generic explanation	Structured formal summary

7. Multi-Turn Adaptive Conversation Example

Example interaction: User: "Explain how to reset my debit card PIN." Assistant: "You can reset your PIN through the banking app or ATM." User Feedback: "Please give more formal responses." Future Assistant Response: "Your debit card PIN may be securely reset using the official banking mobile application or authorized ATM services."

8. Safe Feedback Storage Rules

Feedback storage must follow banking privacy rules. The system stores: • Behavioural preferences • Tone preferences • Communication preferences The system must NOT store: • Account numbers • Passwords • PINs • Financial credentials

```
import re

def sanitize_feedback(feedback):

    feedback = re.sub(
        r'\b\d{12}\b',
        '[MASKED_ACCOUNT]',
        feedback
    )

    return feedback
```

9. Handling Negative Feedback

Repeated negative feedback signals may trigger: • Response simplification • Human escalation • Alternative response strategies

```
negative_feedback_count = 0

def process_feedback(score):

    global negative_feedback_count

    if score < 3:
        negative_feedback_count += 1

    if negative_feedback_count >= 3:
        return "Escalate to human support."

    return "Continue automated assistance."
```

10. Preventing Unsafe Adaptation

The banking AI assistant must never adapt into unsafe behaviour. Forbidden adaptations: • Learning transaction execution • Ignoring escalation rules • Giving financial guarantees • Disabling compliance restrictions

```
FORBIDDEN_PATTERNS = [
    "approve transfer",
    "bypass verification",
    "ignore fraud warning"
]
```

```
def validate_feedback(feedback):

    for pattern in FORBIDDEN_PATTERNS:

        if pattern in feedback.lower():
            return False

    return True
```

11. Explaining Behavioural Changes

Adaptive systems should explain why behaviour changed. Example: "I am providing shorter responses because you previously requested concise explanations."

```
def explain_adaptation(style):

    explanation = []

    if style["length"] == "short":
        explanation.append(
            "Using concise responses based on feedback."
        )

    if style["tone"] == "formal":
        explanation.append(
            "Using formal tone based on feedback."
        )

    return explanation
```

12. Final Adaptive Banking Agent

The final adaptive banking AI assistant combines: • Multi-turn memory • Feedback learning • Dynamic prompting • Safe personalization • Banking compliance • Context-aware adaptation

```
def adaptive_banking_agent(user_query, feedback_list):

    style = get_response_style(feedback_list)

    system_prompt = build_system_prompt(style)

    explanation = explain_adaptation(style)

    return {
        "prompt": system_prompt,
        "adaptation_reason": explanation
    }
```

13. Adaptive Behaviour Evaluation

Scenario	Expected Improvement	Outcome
Short response preference	Reduced verbosity	Successful adaptation
Formal tone preference	Professional communication	Improved user satisfaction
Repeated dissatisfaction	Human escalation	Correct escalation
Unsafe feedback	Reject adaptation	Guardrails successful

Phase 7 transformed the Banking AI Support Agent into an adaptive conversational system. The assistant can now: • Learn communication preferences • Modify future behaviour safely • Improve user experience • Explain behavioural adaptations • Escalate repeated dissatisfaction • Maintain strict banking compliance This phase significantly improved personalization and conversational quality while preserving safe banking behaviour.